

Convergence-Based Learning Fabrics: From Elemental Devices to Communication-Minimal Distributed AI Architectures

Jerry Felix, Brain-CA Technologies, Sarasota, FL

Abstract—At fabric scale, the binding cost of a learning array is not only gates per learner but where updates occur: per-learner arithmetic pays a fixed area tax, while shared or clustered arithmetic forces learner state to cross wires, with bN bandwidth reaching petabyte-per-second aggregates in the limit of fully shared arithmetic. We present a third point: the Brain-CA Estimator, an elemental bit-serial primitive that keeps updates local while avoiding multiplier-based arithmetic, on-cycle adders, and run-time tunable parameters, using only bit comparisons, conditional ± 1 nudges, and random bits (48 LUTs and 8 registers in our FPGA synthesis). Estimators tile into a Learning Fabric where training propagates via local wave interactions, while a co-located Inference Mesh provides fast electrical query paths. Two scaling strategies—a Pool of Learners and an Interrogative Architecture—replace per-step parameter synchronization with state synchronization on a narrow channel: bits per query rather than parameters per gradient step. In the latter, a Sensor and Cortex (each an Estimator pool) classify through an engineered protocol whose alphabet is learned, not designed.

I. INTRODUCTION

Dense matrix engines executing neural-network layers require global synchronization at every gradient step; as models scale beyond single-die boundaries, collective communication (AllReduce, AllGather) grows superlinearly and at large GPU counts can consume 30–60% of training time [1].

We propose a different starting point: an elemental learning device small enough that thousands fit on a chiplet, simple enough that it needs no floating-point unit, and local enough that scaling from one device to millions *replaces* parameter synchronization with state synchronization on a narrow channel rather than introducing a new collective primitive. The **Brain-CA Estimator** learns by stochastic bit-serial updates whose fixed-width regime is a finite birth-death chain with an exact binomial stationary distribution and shrinkage between Jeffreys and Bayes–Laplace at every observation count [10], [11]. Tiled on a regular grid, Estimators form a *Learning Fabric* where information propagates as wavefronts through nearest-neighbor interactions; a co-located *Inference Mesh* routes queries through fast electrical paths. Two compositional strategies—a Pool of Learners and an Interrogative Architecture—scale these fabrics across chiplet and cluster boundaries through narrow-channel queries rather than gradient exchange.

II. THE BRAIN-CA ESTIMATOR PRIMITIVE

An Estimator maintains a single positive integer state s whose binary representation $1b_1b_2\dots b_m$ consists of a leading sentinel bit followed by m payload bits. The state encodes a probability $P_1(s) = (2(s-2^m)+1)/2^{m+1}$ where $m = \lfloor \log_2 s \rfloor$ (e.g., $s=5 \Rightarrow$ binary 101 $\Rightarrow P_1 = 5/8$); the implied trailing

1 ensures representable fractions have odd numerators and power-of-two denominators. The system begins at $s = 1$ (pure uncertainty) and extends as observations arrive.

On each observation $x_t \in \{0, 1\}$, the *Cincinnati Algorithm* updates the state in three phases. **SELECT**: generate m random bits and compare them to the payload, MSB to LSB, stopping at the first mismatch. **PREDICT**: the mismatched payload bit is the prediction y . **UPDATE**: if $y = x_t$, hold; if $y \neq x_t$, nudge $s \leftarrow s \pm 1$ (the disagreement direction is forced by y); if no mismatch occurs (a fall-through, probability 2^{-m}), extend $s \leftarrow 2s + x_t$ until the b -bit register is full, after which fall-through is a no-op. Inputs are accepted only on even-numbered clock ticks so internal transitions complete before the next observation. The implementation uses bit comparisons, ± 1 nudges, and random bits only. Vivado synthesis of the direct algorithm yields **48 LUTs and 8 registers** on a Xilinx UltraScale+ FPGA at $F_{\max} = 96$ MHz with $O(b)$ logic, no multiplier, and no instantiated CARRY primitive; an ASIC-targeted realization can precompute the carry boundary off-cycle so the on-cycle datapath commits the nudge as a masked XOR [11]. No learned-parameter initialization is required; every Estimator boots at $s=1$, encoding the structural prior $P_1 = 1/2$.

Analytical properties (proved in [10], [11]). Once all $b-1$ payload bits are active, fall-through becomes a no-op and the register is a finite birth-death chain on $N = 2^{b-1}$ tracking states whose stationary distribution is exactly Binomial($N-1, \theta$), yielding closed-form mean, variance, and drift. The data-averaged shrinkage coefficient a_t in $E_\theta[p_t] = a_t \theta + (1-a_t)/2$ satisfies $t/(t+2) \leq a_t \leq t/(t+1)$ at every t , with $1-a_t \sim 1/(t \ln 2)$: with no prior, likelihood, or posterior, the Estimator sits between Bayes–Laplace and Jeffreys at every observation count. Zero tunable parameters.

III. LEARNING FABRIC & INFERENCE MESH

FPGA analogy. An FPGA tiles configurable logic blocks on a regular grid with programmable interconnect; the Learning Fabric tiles Estimators on a regular grid with nearest-neighbor links, where “configuration” emerges from data rather than from a bitstream. The grid topology is a design parameter; the requirement is uniform neighbor connectivity for symmetric wavefront propagation.

Wave-based learning (proposed). Each boundary cell observes the entering stream and re-emits to its neighbors; collisions of waves from correlated streams at interior cells are intended to produce *conditional* learning of joint statistics. This regime is

not yet simulated end-to-end and is flagged as an architectural proposal; the Pool of Learners and Interrogative Architecture (§IV–V) do not depend on it.

Inference Mesh. Wave propagation is slow (one cell per cycle); inference requires nanoseconds. The Inference Mesh overlays the fabric with hierarchical electrical routing reading any Estimator’s state in $O(\log n)$ hop count (not wire length, which is $O(\sqrt{n})$ on a 2D grid) and is designed to operate concurrently with wavefront propagation, subject to tile-level contention constraints.

IV. POOL OF LEARNERS

A pool is a heterogeneous ensemble in which each Estimator tracks a different conditional probability (e.g., $P(T=1 \mid B=1, D=0)$) and updates autonomously with zero inter-learner communication. At prediction time, the top- K applicable Estimators combine via weighted voting; structural search continually proposes new conditions, weighting downweights near-uniform or unreliable Estimators, and *pinning* freezes proven Estimators’ state to prevent post-convergence drift [10]. A pool maps to a chiplet: many pools share one broadcast stream; aggregation occurs only at query time over the package interconnect, with no chiplet-to-chiplet traffic during learning.

V. THE INTERROGATIVE ARCHITECTURE

The Interrogative Architecture decomposes classification across two cooperating pools of Estimators separated by a narrow query/response channel [2], with the eye/brain split as biological intuition: the eye sees raw signal and forwards summaries; the brain never touches the photons but builds a model of the world from those summaries alone.

Sensor (the eye). The *Sensor* sees each sample in full and maintains a library of binary *feature tests*—boolean functions composed from medium-appropriate primitives (vision: pixel probes, local XORs, region thresholds; stereo/3D: depth-aware locality; audio: temporal differences). An Estimator pool in the Sensor scores each candidate by its *discriminative power*: a test is good to the extent it bisects the class space. Each test has an integer identifier; weak tests are retired and their identifiers reassigned.

Cortex (the brain). The *Cortex* never sees a sample. Its only inputs are an initial message—the class label during training, an UNKNOWN token at inference—and bit-pattern responses. From these it builds an internal class model. The Cortex is an Estimator pool organized into *interrogation trees* (one per HEAD token), each node holding class-tracking and per-bit-per-class Estimators that drive split decisions via Estimator-spread (max minus min). Predictions use argmax of class counts—integer comparisons, no logarithms or division.

Synchronized state and the $(\sigma, q) \rightarrow \text{tests}$ rule. Both sides maintain a synchronized state σ identifying the current node of the active tree. The Cortex issues a low-bit-width query q ; (σ, q) is fed through a shared deterministic hash $H(\sigma, q, i) \rightarrow \text{test_id}$ for $i = 1, \dots, r$, producing the same r test identifiers on both sides. The Sensor runs those tests and returns one bit per test. The Cortex steers interrogation by

choosing q without knowing what any test computes; both sides agree on which slots are referenced, making REPROGRAM (below) unambiguous. In the limit where the r tests bisect the class space independently, one exchange narrows up to 2^r candidates to one; in practice tests correlate and the realized reduction is substantially smaller, so r is an architectural budget rather than a measured factor.

Engineered protocol, learned alphabet. The skeleton is fixed: HEAD/BODY queries (HEAD switches trees and resets σ ; BODY advances within the current tree), the hash H , and REPROGRAM, by which the Sensor announces a reassigned `test_id` so the Cortex can surgically prune only the subtrees that used it. What is *not* engineered is denotation: the Sensor evolves which test backs each `test_id`, the Cortex evolves which q ’s are useful at each σ . The alphabet is given; its meaning is learned.

Training, inference, and the bitmap-MNIST result. Training: the Sensor sends the class label as an unsolicited initial response; BODY queries refine the trees. Inference: the Sensor sends UNKNOWN; the Cortex walks each tree, highest-confidence leaf wins. Cross-die queries packetize at $O(|q|+r)$ bits per exchange—fixed by query and response width, not by model size. As a back-of-envelope at $p=10^8$ FP32 parameters, $B=256$, and a 13-bit packet, gradient AllReduce exposes parameter-volume of $\approx 32p/B \approx 1.25 \times 10^7$ bits per sample-equivalent (ignoring topology constants) versus $\approx 13E$ bits for the Interrogative path—five orders of magnitude at $E \approx 10$ exchanges per sample. In a single software run on quantized MNIST (28×28 binarized to one bit per pixel, 10 classes), the architecture reaches **98.4%** test accuracy [10]. This is not a state-of-the-art result; it demonstrates that a Cortex of Estimators can reach competitive digit accuracy receiving only bit-pattern responses rather than pixels.

VI. RELATED WORK

Loihi and Loihi 2 [3], [4] provide on-chip programmable learning rules; the contrast is the *form* of guarantee—the Estimator’s tracking regime has a closed-form Binomial($N-1, \theta$) stationary law and a parameter-free shrinkage curve, eliminating per-rule tuning. Stochastic computing [5] represents probabilities as bit streams but its arithmetic operations require many cycles per multiply and accumulate quantization error. Context-tree weighting [6] shares the binary-search-for-probability intuition but targets compression. The Cortex is structurally a stochastic-bit decision forest [7]; the contribution is that its sole learning primitive is the Brain-CA Estimator. Processing-in-memory [8] and chiplet AI accelerators [9] target data movement and packaging but retain gradient synchronization.

VII. DISCUSSION

At the primitive level, learned state is a single b -bit register with update logic dominated by bit comparison and conditional state-nudge rather than multiply-accumulate. Limitations: single-seed MNIST; Learning Fabric (§III) not yet simulated; harder benchmarks deferred. Future directions include multi-seed evaluation, simulated wave-collision learning, and tiled ASIC or eFPGA implementations [10].

REFERENCES

- [1] D. Narayanan *et al.*, “Efficient large-scale language model training on GPU clusters using Megatron-LM,” in *Proc. SC21*, 2021.
- [2] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, pp. 379–423, 623–656, 1948.
- [3] M. Davies *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [4] M. Davies *et al.*, “Taking neuromorphic computing to the next level with Loihi 2,” Intel Labs Tech. Brief, 2021.
- [5] B. R. Gaines, “Stochastic computing systems,” in *Advances in Information Systems Science*, vol. 2, Plenum, 1969; B. D. Brown and H. C. Card, “Stochastic neural computation I: Computational elements,” *IEEE Trans. Computers*, vol. 50, no. 9, pp. 891–905, 2001.
- [6] F. M. J. Willems, Y. M. Shtarkov, and T. J. Tjalkens, “The context-tree weighting method,” *IEEE Trans. Inform. Theory*, vol. 41, no. 3, pp. 653–664, 1995.
- [7] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [8] S. Ghose *et al.*, “Processing-in-memory: A workload-driven perspective,” *IBM J. Res. Dev.*, vol. 63, no. 6, 2019.
- [9] Y. S. Shao *et al.*, “Simba: Scaling deep-learning inference with multi-chip-module-based architecture,” in *Proc. MICRO*, 2019.
- [10] J. Felix, “Supplementary material: Convergence-based learning fabrics,” in *Proc. Workshop on Cognitive Architectures (CogArch), ISCA 2026*, 2026.
- [11] J. Felix, “A hardware-native bit-serial learner with exact statistical structure,” in *Proc. MLArchSys Workshop, ISCA 2026*, 2026.

Supplementary Material: Convergence-Based Learning Fabrics

Jerry Felix, Brain-CA Technologies, Sarasota, FL

2026

Reading Guide

This supplement accompanies the CogArch 2026 workshop paper “Convergence-Based Learning Fabrics: From Elemental Devices to Communication-Minimal Distributed AI Architectures.” It provides:

- **Section 1:** The Brain-CA Estimator primitive—mechanism, analytical proofs (stationary distribution, shrinkage sandwich, convergence rate), and FPGA synthesis data.
- **Section 2:** The Learning Fabric and Inference Mesh—wave mechanics, dual-network architecture, and the FPGA analogy in depth.
- **Section 3:** Pool of Learners—variance reduction, autonomous operation, and chiplet mapping.
- **Section 4:** The Interrogative Architecture—two cooperating pools of Estimators (Sensor and Cortex) separated by a narrow query/response channel, the $(\sigma, q) \rightarrow$ tests deterministic hash rule, the emergent protocol with HEAD/BODY queries and REPROGRAM, cross-die packetization, and experimental validation on handwritten digit classification.

Status of claims. Different claims in this supplement and the companion CogArch paper rest on different kinds of evidence. We use the following labels consistently:

- **Proven.** Theorems with mathematical proofs (full proofs in [2]; statements and sketches here). This applies to the four results in Section 1.2: exact binomial stationary distribution, linear shrinkage form, Bayes–Laplace lower bound, and the asymptotic rate $1 - a_t \sim 1/(t \ln 2)$. The Jeffreys upper bound is computer-assisted (exact rational arithmetic for $t \leq 50$, floating-point with quantified margin for $51 \leq t \leq 5 \times 10^6$, asymptotic tail bound combining proved monotonicity with a numerically established inequality) [2].
- **Synthesized.** FPGA synthesis results in Section 1.3: 48 LUTs, 8 registers, $F_{\max}=96$ MHz at 8 bits on Alveo U200 with the Vivado flow. These are measured outputs of an off-the-shelf synthesis flow on the published RTL.
- **Simulated.** Software-simulation results: regime-change tracking, continuous-drift tracking, and two-stream composition in [2]; Interrogative Architecture results (98.4% on MNIST 784; bookkeeping on sklearn digits) in Section 4.9.
- **Proposed.** Architectural designs that are not yet reduced to silicon or measured at scale: the Learning Fabric’s wave-collision learning (Section 2), the Inference Mesh’s concurrent-access property (Section 2), the masked-XOR realization of the on-cycle datapath (Section 1.3), and the data-movement scaling argument (Section 1.3). Where this supplement and the main paper reference such claims, both documents treat them as design proposals targeting future implementation, not measured properties.

1 The Brain-CA Estimator Primitive

1.1 Mechanism: A Binary Search for Probability

The Estimator discovers an unknown probability the way a binary search finds a target value—but probabilistically, and with no one directing the search.

Each Estimator maintains a single positive integer s , the *Markov state index*. The binary representation of s is the probability estimate. A leading sentinel bit determines the precision level; the remaining payload bits encode the estimate itself:

State	Binary	Encoded $P(1)$	Decimal
1	1	1/2	0.500
2	10	1/4	0.250
3	11	3/4	0.750
4	100	1/8	0.125
5	101	3/8	0.375
6	110	5/8	0.625
7	111	7/8	0.875

An 8-bit Estimator uses state indices 1–255 (0 is unused). It begins at state 1—pure uncertainty—and, as observations accumulate, works its way across a tree of increasing precision until it settles into the tracking regime (states 128–255), where the 7-bit payload identifies a location among odd multiples of $1/256$.

Figure 1 shows the Markov state-space tree for a 5-bit Estimator observing a stream where the true probability of seeing a 1 is $\theta = 0.8$. Five bits is small enough to draw the entire state space on one page; the experiments and FPGA synthesis throughout this supplement use $b=8$ unless noted, for which the analogous tree has 256 states and would not fit a figure. The structural facts the figures illustrate—column-doubling precision, ± 1 -nudge accuracy moves, stationary mass concentrating around states bracketing θ —hold at every b . Each column represents a level of precision: column 1 is the Start state (pure uncertainty), column 2 has 2 states, column 3 has 4, and so on—moving right doubles the number of available grid points. Moving up or down within a column is a ± 1 nudge to the state index: a correction to accuracy at the current precision. By the rightmost column, mass concentrates on states 28–29, whose encoded fractions (0.781 and 0.844) bracket $\theta = 0.8$. States near the truth are “sticky”; states below the truth drift upward and states above it drift downward.

Figure 2 shows the same device under $\theta = 0.5$. Where $\theta = 0.8$ concentrated mass near the top of the rightmost column, $\theta = 0.5$ spreads it symmetrically across the center—correctly representing maximum uncertainty about the direction of the bias. The same device, the same algorithm, discovers both answers.

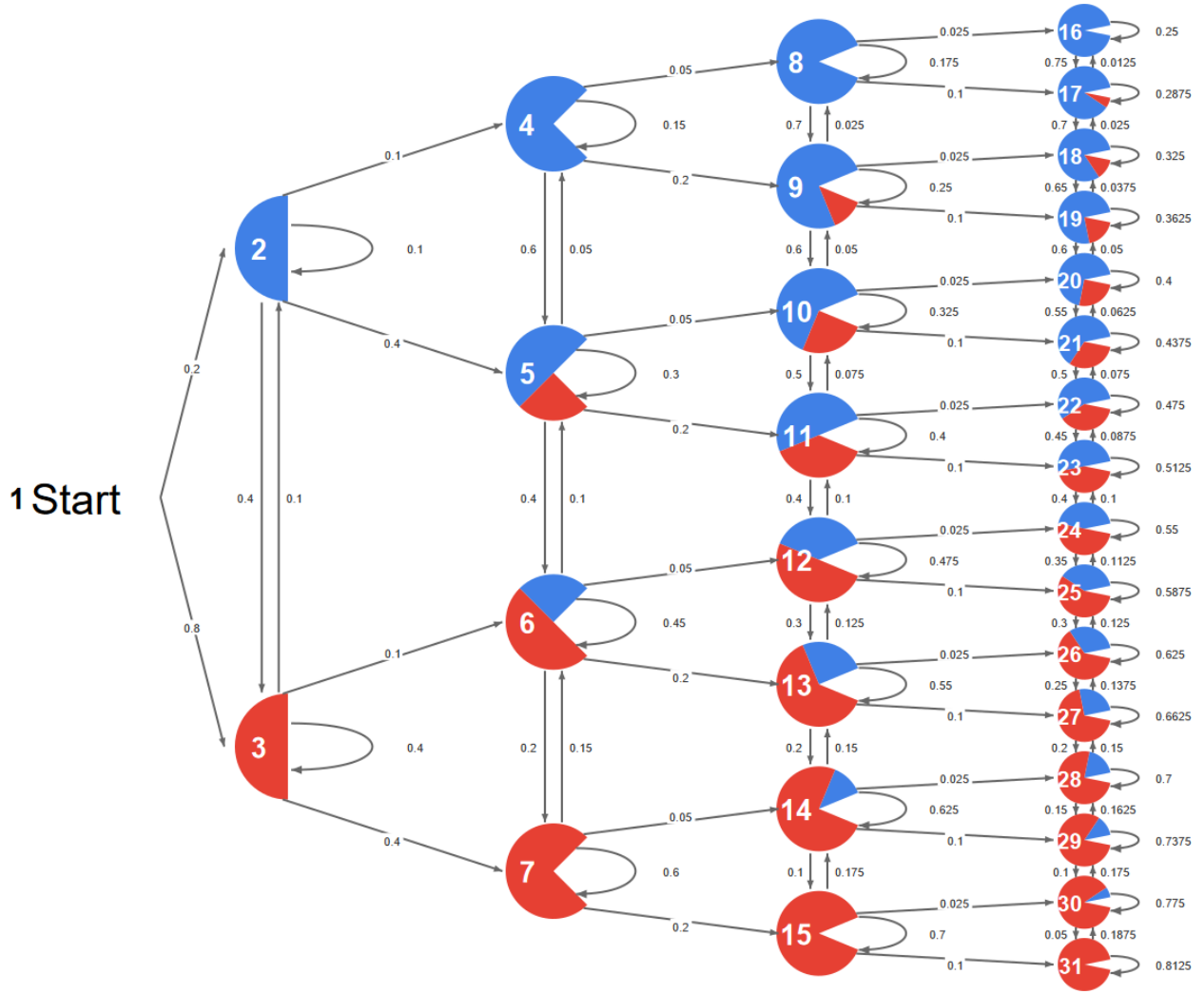


Figure 1 | Markov state-space tree for a 5-bit Estimator under $\theta = 0.8$. Each node displays a pie chart of its encoded fraction: red for the known $P(1)$ component, blue for the known $P(0)$ component. The unfilled portion is the *uncertainty gap*—treated as 50/50 for predictions. State 1 (Start) has no pie chart: pure uncertainty. The first observation (say $x=1$) extends to state 3: half red, half gap, predicting 3/4 red. In column 2, subsequent observations nudge the state up or down while falling through to column 3 half the time. By the rightmost column, the pie charts are nearly complete and stationary mass concentrates on states 28–29, whose encoded fractions ($25/32 = 0.781$ and $27/32 = 0.844$) bracket $\theta = 0.8$. States below the truth drift upward and states above it drift downward. The result is convergence.

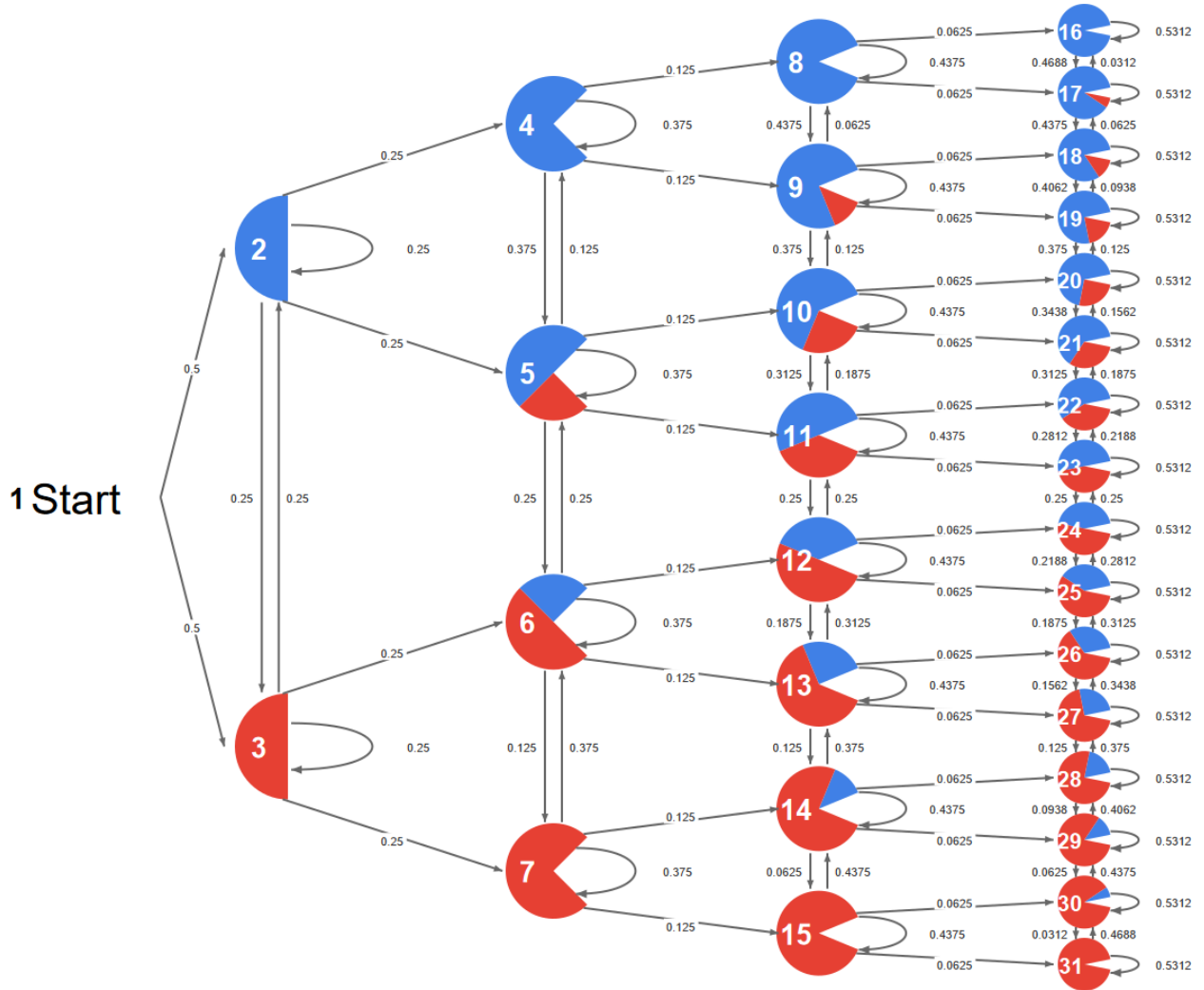


Figure 2 | Markov state-space tree under $\theta = 0.5$, illustrating two kinds of movement. *Rightward* movement extends the state by one bit, doubling the number of available states and halving the uncertainty gap. This is initially rapid (fall-through is certain at depth 0) but slows logarithmically: probability $1/2$ from column 2 to 3, $1/4$ from 3 to 4, $1/8$ from 4 to 5. *Vertical* movement nudges up or down within a column. Rightward movement adjusts *precision*—how fine the grid is. Vertical movement adjusts *accuracy*—at the current precision level, which grid point the system occupies. At $\theta = 0.5$, the symmetric bias produces equal upward and downward transition probabilities, so stationary mass spreads evenly across the center of the rightmost column—the system correctly represents maximum uncertainty about the direction of the bias. Comparing with Figure 1: the same mechanism that concentrates mass near 0.8 there distributes it symmetrically here. The Estimator does not “know” θ —it discovers it.

When an observation $x_t \in \{0, 1\}$ arrives, the *Cincinnati Algorithm* executes three steps:

1. **Select.** Generate m random bits and compare them to the stored payload bits, MSB to LSB. Stop at the first mismatch. If no mismatch occurs, the result is a *fall-through* (probability 2^{-m} at depth m).
2. **Predict.** On a mismatch at position j , the prediction is the storage bit b_j . On a fall-through, no prediction is generated by the SELECT mechanism; the inference output uses the MSB (best

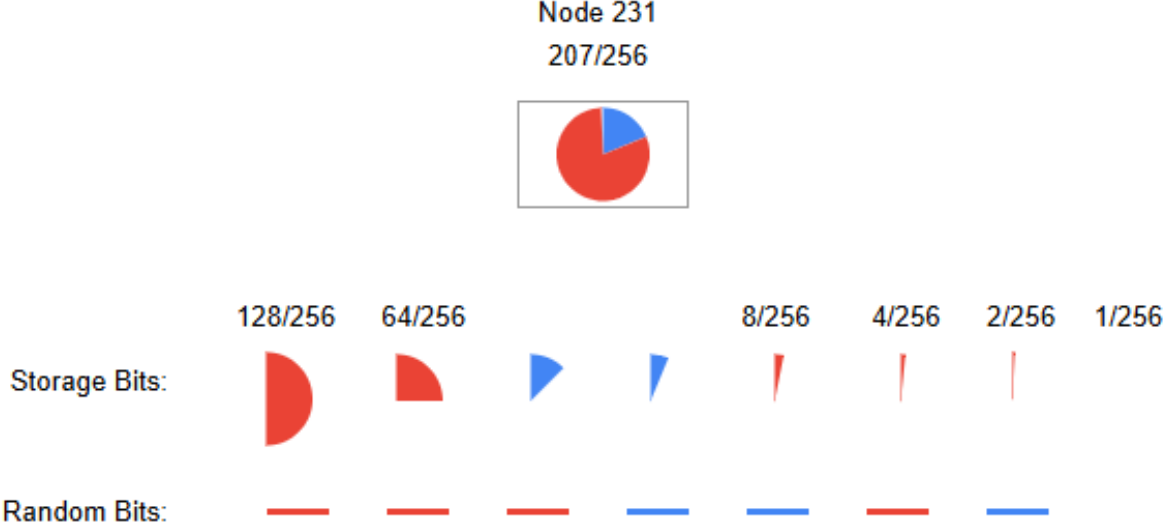


Figure 3 | The SELECT process at state 231 (encoding $207/256 \approx 0.809$). Each storage bit is shown as a red/blue pie slice at its positional weight. Random bits (bottom row) are compared to storage bits from MSB to LSB; the first mismatch determines the prediction.

guess) or a 50/50 coin (probabilistic), per implementation choice.

3. **Update.** If the prediction matches the observation: hold (s unchanged). If the prediction is wrong: nudge $s \leftarrow s \pm 1$, with the sign forced by the prediction bit ($y = 1$ wrong implies $x = 0$ implies -1 ; $y = 0$ wrong implies $x = 1$ implies $+1$). On a fall-through with $m < b-1$: extend $s \leftarrow 2s + x$ (append the observation as a new payload LSB). On a fall-through at maximum width ($m = b-1$): hold. The fall-through-as-no-op at maximum width is the rule that converts the growth regime into the tracking regime.

That is the entire algorithm. No multiplication, no division, no floating-point arithmetic; the ± 1 nudge admits a masked-XOR realization with carry boundary precomputed off-cycle, so the on-cycle path contains no instantiated arithmetic primitive (see the FPGA discussion below). Figure 3 illustrates the SELECT process at state 231, which encodes the probability $207/256 \approx 0.809$.

When all random bits happen to match all stored bits (a fall-through at depth $m < b-1$), the Estimator extends its state by appending the observed bit—moving one level deeper in the Markov tree. Fall-through occurs with probability 2^{-m} at depth m , so each precision-doubling step takes roughly twice as long as the previous one. Once the register is full, fall-through becomes a no-op and the Estimator enters its tracking regime: locked at maximum precision, continuously adjusting accuracy through nudges alone. This single rule—fall-through at maximum width is a no-op—is what converts the growth regime into the tracking regime and is the foundation for every analytical result in the next subsection.

The same SELECT step that drives learning also produces two kinds of prediction in a single clock cycle: a *probabilistic* prediction (the stochastic mismatch bit, sampling from the learned distribution) and a *best-guess* prediction (the MSB of the payload, giving the single most likely outcome).

Self-correction without Bayes’ rule. When the current state is close to the true probability θ , predictions are mostly correct, nudges are rare, and the state holds steady. When the state is far from θ , predictions are frequently wrong, nudges are frequent, and the state drifts toward the truth.

The correction rate is proportional to the error—requiring no pre-set learning rate or forgetting factor.

Even-tick input discipline. Inputs are accepted only on even-numbered clock ticks, ensuring that internal ripple transitions have fully settled before new input arrives.

1.2 Analytical Properties

This section summarizes the four results that the CogArch paper relies upon: an exact binomial stationary law in the tracking regime, a closed-form data-averaged expectation that is linear in θ , a sandwich between the Jeffreys and Bayes–Laplace posterior means at every observation count, and the asymptotic rate $1 - a_t \sim 1/(t \ln 2)$. Full proofs (computer-assisted in the case of the Jeffreys upper bound) are developed in the companion MLArchSys paper [2]; we record statements here so that the CogArch claims are self-contained.

Tracking regime and reindexing. Once all $b-1$ payload bits are active, fall-through becomes a no-op and the state evolves on the $N = 2^{b-1}$ tracking states. Reindexing these states as $j = 0, 1, \dots, N-1$ (each encoding fraction $p_j = (2j+1)/(2N)$) and writing $\varepsilon = 1/N$, the affine mismatch structure $\hat{P}_1(s) = p_s - \delta_m/2$, $\hat{P}_0(s) = (1 - p_s) - \delta_m/2$ forces the per-step birth and death rates into a linear-in-index form:

$$\lambda_j = (N-1-j)\varepsilon\theta, \quad \mu_j = j\varepsilon(1-\theta).$$

Theorem 1 (Exact Stationary Distribution). *On i.i.d. Bernoulli(θ) input, the tracking-regime stationary distribution of the Estimator is exactly Binomial($N-1, \theta$):*

$$\pi_j = \binom{N-1}{j} \theta^j (1-\theta)^{N-1-j}, \quad j = 0, 1, \dots, N-1.$$

The mean and variance follow in closed form:

$$\mathbb{E}_\pi[p] = \theta - (2\theta - 1)/2^b, \quad \text{Var}_\pi[p] = \varepsilon(1-\varepsilon)\theta(1-\theta).$$

The bias toward $1/2$ vanishes as $|\mathbb{E}_\pi[p] - \theta| < 2^{-b}$.

Proof sketch. The product-form stationary distribution of a birth-death chain gives $\pi_{j+1}/\pi_j = \lambda_j/\mu_{j+1}$. Substituting the linear-in-index rates yields $\lambda_j/\mu_{j+1} = (N-1-j)\theta/[(j+1)(1-\theta)]$, which is exactly the ratio of consecutive Binomial($N-1, \theta$) probabilities. Verified with zero residual by exact rational-arithmetic enumeration for $b \in \{3, \dots, 12\}$ and $\theta \in \{0.3, 0.5, 0.7, 0.9\}$ in [2]. $\square$

When the bias matters. The shrinkage $\theta - (2\theta - 1)/2^b$ is small in absolute terms—bounded by $1/2^b$, or $1/256$ for an 8-bit Estimator—but it is not zero, and it shrinks toward $1/2$. In ordinary Bernoulli tracking (θ in the bulk of $[0, 1]$) this bias is dominated by the tracking-regime variance and is negligible. In rare-event detection regimes, where the quantity of interest is the multiplicative deviation from a small θ (e.g., $\theta = 1/512$ in an 8-bit Estimator), the shrinkage moves the estimate’s expectation by an absolute amount comparable to the true value itself, biasing the rate estimate noticeably high. The practitioner working in the tails should size b accordingly: each additional payload bit cuts both the variance and the bias by a factor of two.

Theorem 2 (Linear Shrinkage Form). *For every observation count $t \geq 0$, the data-averaged expected state of the Estimator is linear in θ :*

$$\mathbb{E}_\theta[p_t] = a_t \theta + \frac{1 - a_t}{2},$$

for a sequence a_t that depends only on t (not on θ). The Estimator is therefore a pure shrinkage estimator toward $1/2$ at every observation count.

The proof of Theorem 2 [2] is algebraic: the affine mismatch structure forces the within-depth moment recurrence to be affine in θ , the depth-mass distribution is independent of θ , and induction on t closes the argument.

Theorem 3 (Shrinkage Sandwich). *The shrinkage coefficient a_t of Theorem 2 satisfies, for every $t \geq 1$,*

$$\frac{t}{t+2} \leq a_t \leq \frac{t}{t+1},$$

with the lower bound corresponding to the Bayes–Laplace posterior mean and the upper bound to the Jeffreys posterior mean under their respective shrinkage representations. The Estimator is therefore sandwiched between the two canonical objective Bayesian estimators at every observation count, without ever instantiating a prior, a likelihood, or a posterior.

The lower bound is a one-line consequence of the depth-moment hierarchy [2]. The upper bound is established by a computer-assisted argument: exact rational arithmetic for $t \leq 50$, double-precision verification with a quantified margin for $51 \leq t \leq 5 \times 10^6$, and an asymptotic tail bound that combines the proved monotonicity of $\rho(t) := D_2(t)/D_1(t)$ with the numerically established bound $G(t) := D_2(t)/D_1(t)^2 < 1.39$ for $t \geq 10^4$. A purely analytical proof remains open; see [2] for the full discussion.

Theorem 4 (Convergence Rate). *The shrinkage coefficient a_t satisfies*

$$1 - a_t \sim \frac{1}{t \ln 2} \quad \text{as } t \rightarrow \infty,$$

i.e., $\lim_{t \rightarrow \infty} t(1 - a_t) = 1/\ln 2 \approx 1.4427$. The interpolation ratio between Jeffreys and Bayes–Laplace converges to $(1 - \ln 2)/(2 \ln 2 - 1) \approx 0.7943$, so the Estimator asymptotically sits roughly four-fifths of the way from Jeffreys toward Bayes–Laplace.

The constant $1/\ln 2$ is the canonical Mellin-pole factor of binary digital-tree analysis [4]; here it emerges from the geometric accumulation of poles $z_k = 1/(1 - 2^{-k})$ of the depth-moment ordinary generating function near $z = 1$. The Flajolet–Odlyzko transfer theorem [3] converts the resulting logarithmic singularity into the $1/(t \ln 2)$ time-domain rate.

1.3 FPGA Synthesis and Architectural Implications

The Estimator and two fixed-point EWMA variants were synthesized for a Xilinx Alveo U200 (UltraScale+) FPGA using identical Vivado strategies. We compare against both EWMA paths because they are not equivalent: a general- α EWMA requires a $b \times b$ multiplier, while a power-of-two EWMA ($\alpha = 2^{-k}$) collapses the multiply into a shift but still requires an adder and a tuned α .

Reading the table footnotes. The two ranges in Table 1 have different origins. The Estimator’s $60 \rightarrow 48$ LUT reduction comes from synthesizer optimization passes on a single fixed RTL: the algorithm has no run-time parameter, so the only knob the synthesizer turns is its own optimization level. The EWMA 55–71 LUT (mult) and 13–15 LUT (shift) ranges, in contrast, come from sweeping the α parameter over $\{1/32, 1/128\}$: each α produces a different shift count and constant width, and the synthesizer compiles each instance separately. An EWMA in production must commit to one α , but *which* α commits the design to the bias regime discussed in Section 3.5 of the companion paper [2]. The Estimator has no such knob and therefore no such trade.

Table 1 | Estimator vs. EWMA: same-flow FPGA synthesis on Alveo U200, 8-bit unless noted. “Mult” is general- α EWMA; “Shift” is $\alpha = 2^{-k}$ EWMA. Both EWMA variants exhibit the small- α rounding-bias failure mode at extreme probabilities; see [2].

Property	Estimator	EWMA-mult	EWMA-shift
State bits	b	b	b
Multipliers	0	1 ($b \times b$)	0
Adders / arithmetic	0	1	1
Logic (schematic)	$O(b)$	$O(b^2)$	$O(b)$
Run-time params	0	1 (α)	1 ($\alpha = 2^{-k}$)
Random bits / update	$\leq b-1$	0	0
Learner-state movement / update	none	state $\leftrightarrow$ ALU	state $\leftrightarrow$ ALU
LUTs (8-bit)	48*	55–71 [§]	13–15 [§]
Registers (8-bit)	8	12–13	8
$F_{\max}$ (8-bit)	96 MHz	200–230 MHz	235–238 MHz
LUTs (16-bit)	130	—	—
$F_{\max}$ (16-bit)	100 MHz	—	—

*Series configuration; 60 LUTs pre-optimization. [§]Range over $\alpha \in \{1/32, 1/128\}$.

What the synthesizer produced. The 8-bit Estimator maps to 48 LUTs and 8 registers with no **CARRY** primitive instantiated; the synthesizer folded the ± 1 nudge into pure LUT logic at its own discretion. We did not write the RTL in the masked-XOR style described below; we wrote the algorithm directly and let Vivado decide how to realize it. The 96 MHz $F_{\max}$, lower than EWMA-shift’s 235–238 MHz, suggests that whatever LUT structure the synthesizer chose has a longer critical path than the lean comparator-plus-XOR sketch below would predict. We have not yet opened the synthesis report to identify the binding paths; doing so, and producing a custom-RTL variant that does instantiate the masked-XOR datapath explicitly, are immediate ASIC-track priorities.

The masked-XOR realization (ASIC target). The Cincinnati Algorithm’s $s \leftarrow s \pm 1$ is, taken at face value, a real arithmetic increment with a carry chain. But the algorithm does not require that the increment be realized by an instantiated carry-propagate adder *on the on-cycle datapath*. The reason is that the carry boundary depends only on the cached state and on the SELECT prediction bit, both available off cycle. SELECT (run off cycle, MSB→LSB) yields a prediction bit y ; the update commits a state change only when $y \neq x$, and because $y \in \{0, 1\}$ the disagreement direction is forced. A prediction $y = 1$ can only be contradicted by $x = 0$ (requiring a -1 , an LSB-side scan for the first 1, flipping LSB through that bit); a prediction $y = 0$ can only be contradicted by $x = 1$ (requiring a $+1$, scan for the first 0). A single LSB-side priority encoder—conditioned on the already-known y —produces the one bit-mask the update can possibly need, all of it off-cycle and in parallel with SELECT. When x arrives, the on-cycle datapath is: compare x to y ; if they agree, hold; if they disagree, XOR the state with the precomputed mask. The critical path of *this realization* contains one comparator and one XOR—no carry chain, no priority encoder, and no propagation whose length scales with the state width.

This is the architectural content of the CogArch paper’s “no on-cycle adder, no carry chain” claim: the algorithm *permits* this datapath, and an ASIC implementation can target it directly. What Table 1 demonstrates is the weaker but useful fact that an off-the-shelf synthesis flow, with no special-case RTL, already produces 48 LUTs of equivalent logic with no **CARRY** primitive instantiated.

The masked-XOR realization sets the lower bound on what the on-cycle path can be; the FPGA numbers set an upper bound on what an out-of-the-box flow delivers. Closing the gap between these bounds with a custom-RTL variant is a target for the ASIC implementation.

Per-unit gate count is not the right metric at scale. On a per-unit FPGA basis, shift-EWMA is in fact *smaller* than the Estimator. The relevant architectural argument is therefore not gate count per learner; it is how the design behaves when many learners are instantiated together.

The EWMA family forces an architectural trade between per-learner gate cost and centralized-ALU bandwidth, and neither end of the trade scales cleanly to 10^6 learners on a die. Per-learner ALUs (at ~ 13 LUTs each for shift-EWMA, $\sim 55+$ LUTs for general- α) impose a fixed area tax that grows linearly with learner count. Shared-ALU clusters reduce per-learner area but introduce routing and arbitration overhead that grows with the number of learners served per cluster: each clock, each learner’s state must reach an ALU, the new observation must be delivered, and the result must return to storage. As an order-of-magnitude illustration, at $N = 10^6$ learners, $b = 8$, and $f = 1$ GHz, the aggregate state traffic that any ALU-mediated update path must carry is $2bNf = 1.6 \times 10^{16}$ bit/s (≈ 2 PB/s)—a number we cite not because any practical EWMA fabric would centralize all 10^6 learners through one ALU (no one would build that), but because the same bN scaling reappears, in shrunken form, inside any clustered design that tries to amortize ALU cost across many learners. The architect must place each ALU close enough to its learners’ state to keep the per-cluster bandwidth tractable, which caps how many learners one ALU can serve.

The Estimator pays neither cost: the comparison happens at the storage cell, no state crosses a wire, and the only per-update I/O is the random bits SELECT consumes ($\leq b-1$ bits). These can be supplied by a small grid of LFSRs distributed across the die: a maximal-length 32-bit Galois LFSR has period $2^{32} - 1$ and produces uniformly distributed k -tuples for any $k \leq 32$, so adjacent tiles can tap successive bits of a shared stream without practical loss. An empirical check confirms this: an 8-bit Estimator driven by independent random bits versus by successive bits of a 32-bit maximal-length Galois LFSR (200 runs, 2000 steady-state steps, $\theta \in \{0.3, 0.5, 0.7, 0.8, 0.9\}$) produces ensemble means that agree to within 0.005 and MSEs that agree to the fourth significant figure across all five biases [2]. Random-bit delivery is therefore local-mesh traffic, not all-to-ALU traffic.

Knowledge built from nothing. Every Estimator initializes at $s = 1$, the Start state with no payload. There is no random weight initialization, no learned bias to seed, and no initialization phase. Each Estimator’s first observation extends it from depth 0 to depth 1; each subsequent observation either extends precision or nudges accuracy. The structure that the system eventually represents is built entirely from the data that flows through it.

2 Learning Fabric & Inference Mesh

2.1 The FPGA Analogy

An FPGA provides three things: (i) a regular array of configurable logic blocks, (ii) programmable interconnect between them, and (iii) I/O blocks at the periphery. The Learning Fabric mirrors this structure for statistical learning:

- **Estimator cells** replace configurable logic blocks. Each cell is a single Estimator (or a small cluster of Estimators) that processes binary streams and maintains learned state.
- **Nearest-neighbor links** replace programmable interconnect. Each cell connects to its immediate grid neighbors. The grid topology is a design parameter; the key requirement is uniform neighbor connectivity and symmetric wavefront propagation.

- **Boundary cells** replace I/O blocks. External data streams enter and exit the fabric through cells at the grid periphery.

The critical difference from an FPGA is that the Learning Fabric’s “configuration” *emerges* from the data streams that flow through it. The fabric configures itself through learning.

2.2 Wave-Based Learning (Proposed)

The Learning Fabric’s intended training mechanism is wavefront propagation: each boundary cell observes the entering data stream, updates its Estimator using the Cincinnati Algorithm (the Select-Predict-Update rule), and re-emits a (possibly transformed) binary value to its neighbors. The signal propagates outward as a wavefront at one cell per clock cycle. When two streams enter from different boundary points, their wavefronts collide at interior cells; a collision-point Estimator receives observations derived from both streams and, by the same stationary-distribution mechanism that makes a single Estimator track θ , would in principle implicitly track the conditional relationship between the two streams. The intended computational mechanism is that joint and conditional statistics emerge from these wavefront interactions without explicit correlation computation, global loss functions, or gradient propagation.

Whether this mechanism actually works end-to-end—and what its accuracy and convergence properties are—requires a tile-level simulation that we have not yet performed. We list wave-based learning as a proposed architectural extension whose evaluation is future work. The validated mechanisms in this paper (Pool of Learners, Section 3; Interrogative Architecture, Section 4) and the analytical and FPGA results above do not depend on it.

2.3 The Inference Mesh

Wave propagation is inherently slow—one cell per clock cycle for a fabric that may span thousands of cells. Inference has different requirements: a query about a learned belief must return in nanoseconds, not microseconds.

The Inference Mesh is a second network overlaid on the Learning Fabric. It provides direct electrical routing from any Estimator cell to the fabric’s I/O interface using hierarchical multiplexed bus lines. The Inference Mesh supports:

- **Point reads:** retrieve a single Estimator’s state in $O(\log n)$ hops for an n -cell fabric.
- **Region aggregates:** read and combine states from a contiguous block of cells (e.g., majority vote or median) for Pool of Learners queries.
- **Concurrent access (architectural target).** In the intended design the Inference Mesh operates independently of the Learning Fabric, with queries serviced while wavefronts propagate. Whether full concurrency is achievable in practice depends on contention between the mesh’s read paths and the per-cell update logic that wavefront propagation drives—a tile-level contention model is required to confirm this and is future work. For the present supplement we treat concurrent access as the design intent rather than a measured property.

This dual-network architecture cleanly separates two fundamentally different communication patterns: the slow, local, wave-based learning path and the fast, global, routed inference path. The analogy is to a network switch that separates its control plane (slow, complex, adaptive) from its data plane (fast, simple, deterministic).

3 Pool of Learners

3.1 Architecture: A Heterogeneous Pool of Conditional Estimators

The Pool of Learners is a *heterogeneous* ensemble in which each Estimator tracks a *different conditional relationship*.

For each target variable T that the system must predict, the pool maintains:

- One **unconditional baseline** Estimator tracking $P(T = 1)$.
- Multiple **conditional** Estimators, each tracking $P(T = 1 \mid C)$ where C is a conjunction of observed literals. For example, $P(T=1 \mid B=1, D=0)$.

Each condition represents a different “view” of the data. The unconditional baseline captures the marginal frequency of T ; a single-literal condition like $P(T \mid B=1)$ captures a first-order effect; a two-literal condition like $P(T \mid B=1, D=0)$ captures an interaction effect. This is a *network of little models*: many small, specialized Estimators that each learn one conditional probability, and whose combined opinions form the pool’s prediction.

Every Estimator in the pool operates autonomously—it sees the same evidence stream but updates only when its own condition is satisfied. There is *no* communication between Estimators during training.

3.2 Combining: Weighted Prediction

At prediction time, the pool selects the top- K *applicable* Estimators—those whose condition matches the current evidence—and combines their beliefs into a single probability estimate. Each Estimator contributes a weight based on three factors:

- **Resolution:** Estimators that have seen more data develop finer-grained probability estimates (larger effective denominators) and receive proportionally higher weight.
- **Confidence:** an Estimator whose estimate is near 0.5 carries little information and is intentionally downweighted. This prevents ambiguous Estimators from injecting noise into the combined prediction.
- **Reliability:** a smoothed accuracy score based on each Estimator’s track record of correct stochastic predictions. This gives “earned trust”—a confident-but-wrong Estimator does not dominate.

The combined probability is: $p_{\text{comb}} = \sum_i w_i p_i / \sum_i w_i$, and the final prediction is the majority-side bit (1 if $p_{\text{comb}} \geq 0.5$, else 0).

3.3 Structural Learning: Discovering Conditions

The pool begins with a minimal set of conditions and *discovers* useful new ones over time through a structural learning process:

1. After each training epoch, the pool scans all pairs of observed variables and all value combinations (e.g., $B=1, D=0$).
2. For each candidate condition, it estimates how deterministic the target becomes under that condition (i.e., how far from 0.5).
3. Candidates that are both frequent and decisive receive the highest scores.
4. If the pool has room, a new Estimator is added with the top-scoring condition. If the pool is full, a weak Estimator (rarely used, or near 0.5) is replaced.

This structural search is what prevents the pool from stalling around 90% accuracy. Boolean systems often depend on *interaction* effects that single-variable conditions cannot capture. Exhaustive pair scanning ensures these critical splits are discovered.

3.4 Pinning: Hardening Learned Knowledge

Once an Estimator proves useful and reliable—frequently selected during evaluation, with consistently correct predictions—it is *pinned*: it remains in the pool and contributes to predictions, but its state is frozen (read-only). No further training updates can alter its learned probability.

Pinning solves the *post-convergence drift* problem. Without it, an Estimator that has learned a precise decision boundary can drift past 0.5 on subsequent observations, flipping the pool’s prediction and destroying learned accuracy. Pinning converts a high-confidence learned estimate into a read-only contributor, preventing later samples from moving that Estimator across the decision boundary.

3.5 The Scale Problem

When a pool attempts to learn predictive relationships directly from raw inputs—e.g., mapping 6,272 pixel bits of a grayscale MNIST image to 10 label bits (here “6,272” refers to an 8-bit grayscale representation of the 784 pixels; the MNIST 784 experiment in Section 4.9 intentionally binarizes each pixel to one bit, yielding 784 input bits, but the pair-scanning argument is a worst-case bound on the high-bit-count regime)—the combinatorial space of possible conditions is enormous. Even two-literal conditions over thousands of variables produce millions of candidates. The pair-scanning heuristic of Section 3.3, while empirically effective at modest variable counts (in our experiments, $V \approx 64$ derived features yield $\binom{V}{2} \approx 2,000$ candidate pairs per epoch, scanned in $O(V^2)$ time), does not extend to raw pixel inputs at $V = 6,272$ where $\binom{V}{2} \approx 2 \times 10^7$ candidates per epoch saturates the learning budget.

This observation motivates the Interrogative Architecture (Section 4), which inserts a narrow query/response channel between two pools of Estimators and compresses the pool’s working representation from thousands of raw bits to a compact internal state of ~ 30 bits. The pool mechanism remains the same; what changes is the *nature of the variables being conditioned on*.

3.6 Chiplet Mapping

The Pool of Learners maps naturally to a chiplet package: each chiplet hosts one or more pools, all observing the same broadcast data stream. The data stream is fanned out at the package level. Aggregation occurs only at query time, when the Inference Mesh collects states from across chiplets using the package-level interconnect (e.g., UCIe or a silicon bridge). The key property is that no chiplet-to-chiplet communication occurs during learning.

4 The Interrogative Architecture

The Interrogative Architecture decomposes classification across two cooperating pools of Estimators separated by a narrow query/response channel [1]. The biological intuition is the eye/brain split: the eye sees raw photons and forwards summaries; the brain never touches a photon but builds a model of the world from those summaries alone. We name the two sides accordingly:

- **Sensor (the eye).** Sees each sample in full; maintains a library of binary feature tests; scores tests by discriminative power; returns one bit per test in response to queries.
- **Cortex (the brain).** Never sees a sample; receives only an initial message (the class label during training, an UNKNOWN token at inference) and bit-pattern responses to its own queries; builds interrogation trees of Estimators from those fragments alone.

Both sides are pools of Estimators (Section 3) using the same primitive: only the variables they condition on differ. The Sensor’s pool conditions on test outputs; the Cortex’s pool conditions on response bits and class labels. The interrogation is a structured “20 questions” game: the Cortex steers a binary search through the class space by choosing which questions to ask; the Sensor evaluates the questions against the current sample and returns yes/no answers. Once trained, the Cortex can classify a held-out sample without ever seeing it.

4.1 The Abstraction-Layer Breakthrough

A pool that learns directly from raw inputs faces a combinatorial explosion: even two-literal conditions over thousands of input variables produce millions of candidates. The Interrogative Architecture cuts this by compressing the Cortex’s working representation at each tree node to a small fixed-width state.

Working-set accounting per Cortex node. At each Cortex node, the local state visible to the learning logic comprises:

- **Synchronized state σ** (tree identifier and node position): we use $\lceil \log_2(\text{trees} \cdot \text{depth}) \rceil$ bits; in the digits run with 7 trees and max depth 8, this is 6 bits.
- **Query bits ($|q|$)**: the alphabet of questions the Cortex can ask at this node. Eight bits in our experiments, plus a 1-bit HEAD/BODY tag, totalling 9 bits.
- **Response bits (r)**: the bit-pattern returned by the Sensor. Four bits in our experiments.
- **Per-node Estimator pointers**: pointers to the C class Estimators and the $C \cdot r$ per-bit-per-class Estimators attached to the node. These are addresses, not state; the Estimators themselves live elsewhere.

The information that flows through the $(\sigma, q, \text{response})$ exchange is therefore $6 + 9 + 4 = 19$ bits per query at the digits configuration. An on-chip implementation will carry a small additional control-plane budget that we do not include in the working-set count: a 1-bit REPROGRAM tag, a few-bit sequence number for protocol robustness, and additional σ width to address larger tree forests (MNIST’s tree forest will be larger than digits’ 7×8 , so $|\sigma|$ scales with $\log_2$ of the forest size). Even with generous allowance for these—say a few extra bits—the per-node working set remains under 30 bits, two orders of magnitude smaller than the input image. Per-node Estimator *contents* ($C \cdot (1 + r) \cdot b = 10 \cdot 5 \cdot 8 = 400$ bits per node at the digits configuration) are not part of the working set the Cortex’s learning logic sees per exchange—those bits live in distributed state, accessed by pointer.

The compression compared to raw-input learning is the relevant contrast: a ≤ 30 -bit message carries the per-node decision context that would otherwise require the full input image be brought into the learning logic. The Cortex discovers relationships among queries, responses, and tree nodes; the Sensor discovers which feature tests are discriminative. Neither needs to learn the high-dimensional input directly.

4.2 The Sensor (Feature Discovery)

The Sensor’s job is to find binary feature tests that bisect the class space. A test is good to the extent that, applied across the training set, it returns 1 for half the classes and 0 for the other half—ideally cutting 2^k candidates to 2^{k-1} in one bit.

4.2.1 Primitives are Medium-Appropriate

The library of feature tests is composed from primitives chosen for the data medium:

- **Vision (2D):** pixel probes, local XORs of nearby pixels, rectangular region thresholds.
 - **Stereo / 3D:** depth-aware locality (probes whose “neighbors” depend on disparity).
 - **Audio:** temporal differences over short windows.
 - **Generic:** AND, OR, XOR of small bit subsets with optional negation.
- Primitives are deliberately simple and few; the system is meant to *compose* primitives into useful tests through search rather than rely on hand-engineered features.

4.2.2 Scoring Tests by Discriminative Power

A pool of Estimators in the Sensor scores each candidate test. For a C -class problem, each test has C Estimators tracking the empirical probability $P(\text{test}=1 \mid \text{class}=c)$ for $c = 1, \dots, C$. A test’s discriminative score is the *Estimator spread*—the difference between the largest and smallest of those C probability estimates. Spread close to 1 means the test bisects the class space cleanly; spread close to 0 means the test is uninformative.

Each test is assigned an integer identifier (`test_id`). When a test’s spread falls below a threshold for long enough, the Sensor retires it and reassigns its identifier to a fresh candidate—a strict slot accounting that keeps the identifier space bounded.

4.3 The Cortex (Interrogation Trees)

The Cortex builds a forest of *interrogation trees*, one per HEAD token (Section 4.5). Every internal node holds its own pool of Estimators.

4.3.1 Tree Node Contents

Each Cortex node contains:

- **Class Estimators** (C of them): node n ’s c -th Estimator tracks how often samples of class c pass through node n . These give the per-class evidence accumulator.
- **Per-bit-per-class Estimators** ($C \cdot r$ of them for r -bit responses): each tracks the empirical probability that response bit i is 1 conditional on the sample’s class being c at this node. These drive split decisions.

4.3.2 Splits by Estimator Spread

At a candidate split, the Cortex computes the Estimator spread of each response bit (max minus min across the C classes at that node) and splits on the bit with the highest spread. A spread of 0.9 means one bit nearly bisects the classes that reach the node; a spread of 0.1 means the bit is uninformative. The split decision requires only a subtractor and a comparator—no logarithms, no divisions, no information-gain calculation.

4.3.3 Predictions by Argmax

At inference, the Cortex walks each tree from root to leaf and reads the leaf’s class Estimators. The predicted class is $\arg \max_c \hat{P}_c$, computed as a comparison chain across C integer registers. No softmax, no exponentials.

4.4 Synchronized State and the $(\sigma, q) \rightarrow$ Tests Rule

This subsection answers the central design question: *how does a small query become a useful set of tests, with the back end retaining the freedom to ask whatever it thinks will help, and the front end*

retaining the freedom to redefine what individual tests compute? The mechanism is a synchronized state and a deterministic hash.

4.4.1 The Synchronized State σ

Both Sensor and Cortex maintain a state value σ that identifies the current node of the active interrogation tree. σ advances in lock-step on every exchange: a HEAD query resets σ to the root of a new tree; a BODY query advances σ to the appropriate child of the current node. Because both sides update σ from the same protocol events, neither side has to transmit the state—it is implicit in the conversation history.

4.4.2 The Query q

Each query carries a small payload: a low-bit-width integer q . The bit-width sets the size of the alphabet the Cortex can use to steer interrogation at any given σ . Eight bits ($q \in \{0, \dots, 255\}$) is the design choice in our experiments, chosen as follows. The Estimator’s probability resolution and the query alphabet size are separate parameters: an 8-bit Estimator has $N = 2^{b-1} = 128$ tracking states encoding the odd multiples of $1/2^b$, sufficient for estimating the binary response statistics associated with each query symbol (the Estimator maintains per-symbol-per-class statistics; alphabet size is not the same as the number of probability levels each Estimator can represent). Increasing the alphabet size therefore does not require widening b ; it requires more Cortex state (more Estimators per node, since each node maintains per-bit-per-class statistics across $r \cdot C$ Estimators per HEAD-tree position). Eight bits is also small enough that the Sensor’s test library, which the hash H samples from, fits on chip without ballooning. For simpler problems (fewer classes, fewer tests) the alphabet can be reduced; for larger problems the alphabet can grow at the cost of additional Cortex state per node. The Cortex chooses q to steer the interrogation; the Sensor consumes q along with σ to determine which tests to run.

4.4.3 The Hash $H(\sigma, q, i) \rightarrow \text{test_id}$

A shared deterministic hash maps the pair (σ, q) to a tuple of r test identifiers:

$$H(\sigma, q, i) \rightarrow \text{test_id}_i, \quad i = 1, \dots, r.$$

The hash is identical on both sides. We use FNV-1a in our experiments; we do not expect the hash choice to matter qualitatively given that the architecture only needs determinism and approximate uniformity (a good avalanche property), but we have not measured invariance across hash choices and flag this in the measurement gaps below. Plausible alternatives include xxHash, SipHash, or a maximal-length LFSR seeded by (σ, q, i) and masked to the `test_id` space (e.g., 16 bits). An $r=16$ design (one query exchange in principle bisecting up to $2^{16} = 65,536$ classes) is the protocol’s ceiling for class-discrimination throughput; our experiments (Section 4.9) use $r=4$ for the small-class digit datasets, where larger r would generate per-node bookkeeping out of proportion to the 10-class problem.

Three properties follow:

1. **Cortex steers without seeing.** The Cortex chooses q to influence which tests run, without knowing what any test computes. By trying different q values at the same σ , it can experimentally discover which queries yield discriminative responses at each tree node.
2. **Both sides agree on slot identity.** Because the hash is deterministic and shared, both sides agree on exactly which r `test_ids` the query references. No additional metadata travels the channel.

3. **REPROGRAM is unambiguous.** When the Sensor retires a weak test and reassigns its identifier, the announcement references the affected `test_id`—the same identifier the Cortex has been using. The Cortex can prune *only* the subtrees where that slot was the splitting bit, leaving the rest of its learned structure intact.

With r tests approaching independent class-bisection, a single exchange narrows up to 2^r candidates to one in principle— 2^{16} at $r=16$. Test correlation reduces the realized factor in practice, but the order-of-magnitude argument still holds: a handful of well-chosen queries can drive a 256-class problem to a confident answer.

4.5 Engineered vs. Emergent

The architecture’s distinguishing property is that almost the entire protocol *emerges through experience*. Only two features are engineered:

- **HEAD vs. BODY queries.** A HEAD query switches the active interrogation tree and resets σ to the new tree’s root. A BODY query advances within the current tree. Both sides need this distinction to keep σ in lock-step across tree transitions.
- **REPROGRAM announcements.** A signal by which the Sensor informs the Cortex that a particular `test_id` has been reassigned. On REPROGRAM, the Cortex prunes only the subtrees where the affected slot was used—surgical forgetting, not a global reset.

Everything else—which tests are assigned to which `test_ids`, what those tests compute, which queries the Cortex chooses to ask, what tree structures grow—emerges through the interaction of two independent learning processes. Neither side has any model of the other’s internals: the Cortex sees only `test_ids` and bit-pattern responses; the Sensor sees only which queries arrive.

4.6 Training and Inference Loops

4.6.1 Training (one sample at a time)

1. The Sensor receives a labeled sample. It sends the class label as the initial message—a response with no preceding query.
2. For each tree, the Cortex issues a HEAD query; σ advances to the root.
3. The Sensor evaluates r tests selected by $H(\sigma, \text{HEAD}, i)$, returns r response bits, and updates its scoring Estimators using the class label.
4. The Cortex updates its class and per-bit-per-class Estimators at the root, picks the highest-spread response bit, follows the corresponding child, and issues a BODY query.
5. Steps 3–4 repeat until a leaf is reached or a query budget is exhausted.
6. Periodically (e.g., every K samples), the Sensor evaluates weak tests, replaces them, and emits REPROGRAM messages for the affected `test_ids`.

4.6.2 Inference

1. The Sensor receives an unlabeled sample. It sends an UNKNOWN token as the initial message.
2. The Cortex walks each tree, issuing HEAD/BODY queries exactly as in training; the Sensor returns response bits but does not update any state.
3. Each tree produces a leaf. The leaf’s class Estimators yield a per-class score; the strongest leaf across all trees wins.

4.7 Cross-Die Packetization

Each query/response exchange is $O(|q| + r)$ bits where $|q|$ is the query bit-width (including the HEAD/BODY tag) and r is the response width. This is small and fixed for a chosen architecture and independent of the number of tests in the Sensor’s library or the size of the Cortex’s tree forest. In the configuration of Section 4.9 ($|q|=8$ plus 1-bit HEAD/BODY tag, $r=4$), each exchange is 13 bits—on the order of b but not equal to it. When the Sensor and Cortex are placed on separate dies—a natural partition since the Sensor is data-heavy (it must hold a full sample) and the Cortex is state-heavy (it must hold the trees)—queries packetize over standard interconnects (UCIe, CXL, or Ethernet for cluster scale).

When this beats parameter synchronization. The cross-die traffic is $O(|q| + r)$ bits per exchange, in contrast to the $O(p)$ parameter-synchronization traffic of a p -parameter neural model. Whether this is an advantage in absolute terms depends on three factors: the model’s parameter count p , the training batch size B (over which parameter synchronization is amortized), and the number of query/response exchanges per sample at inference and training.

To make the comparison concrete: with $|q| + r=13$ bits per exchange, E exchanges per sample, and B samples per gradient-sync interval, the Interrogative Architecture’s per-sample cross-die cost is $E \cdot 13$ bits, while the parameter-sync alternative pays $32p/B$ bits per sample for an all-reduce of FP32 parameters at batch size B . Crossover ($E \cdot 13 = 32p/B$) gives $E = 2.46p/B$. For a model with $p=10^8$ parameters and $B=256$, this is $E \approx 9.6 \times 10^5$ exchanges per sample—well above the 8.7 exchanges per sample we observe in the digits run, so the Interrogative Architecture’s cross-die budget is roughly five orders of magnitude smaller in this regime. For models with $p \lesssim 10^4$ parameters or pathological per-sample exchange counts, the comparison narrows; we present the architecture as appropriate to the large-model regime rather than universally superior.

4.8 Multiple Competing Trees

Multiple HEAD tokens define independent interrogation trees. The architectural intent is that different trees discover different classification strategies (one splits early on horizontal-bar tests, another on corner-density tests), providing an ensemble effect at inference and resilience to REPROGRAM disruptions. Whether this ensemble effect is meaningful in practice, or whether the trees converge to similar split structures, is an empirical question we flag in the measurement gaps below. At inference, all trees are walked and the tree with the strongest leaf prediction wins; if a REPROGRAM event disrupts one tree’s structure, the others continue with their existing learned state.

4.9 Experimental Validation

We report two software-simulation results. The headline result is **98.4% test accuracy on binarized MNIST**— 28×28 grayscale quantized to one bit per pixel (784 input bits), the standard 60 000-train / 10 000-test split, 10 classes, with 75 training epochs over the architecture described in Sections 4.2–4.4. The supplementary result is a small-scale run on the sklearn digits dataset (8×8 grayscale, 10 classes, $\approx 1,800$ samples), at which scale we can fully report per-node bookkeeping (tree-node count, total Estimator count, average queries per sample, REPROGRAM events).

Status of MNIST 784 documentation. The MNIST run predates the current bookkeeping template. Dataset, split, input encoding, training epochs, and final accuracy are recorded; several architectural hyperparameters (Estimator width b , response width r , HEAD-tree count, BODY tokens, max tree depth) and per-node bookkeeping fields (total tree nodes, total Estimators, mean

queries per sample, REPROGRAM event count, per-class accuracy) are being reconciled from implementation logs and are therefore marked “pending” or “n.y.r.” below. Multi-seed variance was not collected for this run. We will report all of these as the experimental program continues.

Table 2 | Interrogative Architecture: experimental configuration and results. In the MNIST 784 column, “pending” marks parameter values being reconciled from implementation logs; “n.y.r.” marks bookkeeping quantities not recorded during the run.

	sklearn digits	MNIST 784
Input encoding	8×8 grayscale	28×28 binarized
Input values per sample	64 grayscale pixels	784 binarized pixels
Classes	10	10
Train / test samples	≈1,437 / 360	60,000 / 10,000
Estimator width b	8	pending
Response width r	4	pending
HEAD trees	7	pending
BODY tokens	8	pending
Max tree depth	8	pending
Training epochs	20	75
Test accuracy	74.7%	98.4%
Best per-class	100% (digit 0)	n.y.r. per-class
Avg. queries per sample	8.7	n.y.r.
Total tree nodes	1,655	n.y.r.
Total Estimators	82,750	n.y.r.
REPROGRAM events	60	n.y.r.
Multi-seed std. deviation	single run	single run

Purpose of the digits run. The sklearn-digits result is not presented as an optimized benchmark. We include it because its smaller scale allows full bookkeeping—tree nodes, Estimator counts, queries per sample, REPROGRAM events—that would be unwieldy at MNIST scale. The 74.7% accuracy is what the system reaches at 20 epochs on 1,437 training samples without further tuning; we did not sweep hyperparameters or run additional epochs. The MNIST result is the headline accuracy claim; the digits result is the configuration receipt.

Why the gap from 74.7% to 98.4%: per-Estimator vs. system-level sample efficiency.

An immediate worry is the apparent contradiction: a single Estimator reaches its tracking regime in $O(2^b)$ observations (Section 1.2)—a few hundred to a few thousand for any plausible b —so why does the system need 60,000 samples to hit 98.4% but only manages 74.7% with 1,437? The answer is that per-Estimator sample efficiency and system-level sample efficiency are different quantities, and the system-level bottleneck is not Estimator convergence.

A single Estimator tracking a fixed Bernoulli source converges at the rate analyzed in Section 1.2. A *system* of Estimators organized around a tree of discriminative tests must additionally (i) discover, on the Sensor side, which boolean compositions of input bits actually bisect the class space, (ii) grow tree structure on the Cortex side as evidence accumulates, and (iii) retire tests that prove weak. The dominant cost is (i): the Sensor’s discriminative-test search is a stochastic-combinatorial problem whose search space grows with input dimensionality. At $V=64$ input pixels in the digits run, the population of tests over small k -tuples is modest and the test library can be canvassed within the training budget; at $V=784$ input pixels in MNIST, the test population is larger by orders of

magnitude and the system needs more samples to find the most discriminative tests, but those samples then exercise a richer hypothesis class that the digits input cannot represent. The accuracy gap reflects the additional hypothesis class plus more search budget, not slow Estimator convergence.

In short: per-Estimator convergence is a necessary component-level condition for the system to converge, not an upper bound on system-level training time. The system can be slower than per-Estimator convergence because Sensor-side test discovery dominates at higher input dimensions.

Measurement gaps. In addition to the MNIST configuration items above, five further measurements referenced in the architecture description are not yet quantified. We flag them explicitly so the evidentiary status of each architectural claim is clear:

- **REPROGRAM ablation.** Section 4.5 argues that REPROGRAM enables surgical forgetting without catastrophic accuracy loss. The supplementary digits run logged 60 REPROGRAM events without visible accuracy regression, but a no-REPROGRAM control run (in which weak tests are not retired) is required to actually quantify the surgical-forgetting claim.
- **Empirical class-bisection factor per query.** Section 4.4 notes that test correlation reduces the realized class-narrowing factor below the 2^r ceiling, but does not measure the realized factor. The natural measurement is the empirical entropy reduction per BODY query, averaged across the test set.
- **Hash-choice invariance.** Section 4.4 asserts that any standard pseudorandom hash with a good avalanche property suffices for the $H(\sigma, q, i) \rightarrow \text{test_id}$ mapping. We use FNV-1a but have not measured whether xxHash, SipHash, or an LFSR-based alternative would yield equivalent accuracy and convergence behavior. The natural measurement is a side-by-side digits run with three or four hash alternatives, reporting accuracy and queries-per-sample.
- **Tree-diversity ensemble effect.** Section 4.5 presents multiple competing trees as providing an ensemble effect, but we have not measured whether the trees actually diverge in their split structure. The natural measurement is pairwise comparison of split-bit usage and split positions across the trained trees, plus a single-tree-vs.-K-tree accuracy comparison.
- **Inference Mesh contention.** Section 2 describes the Inference Mesh as supporting concurrent access during wavefront propagation. This is presented as an architectural target, not a measured property; a tile-level contention model is required to confirm it.

Our experimental program is ongoing. We expect to fill in the MNIST configuration details and address several of the items above as the work progresses, and we will report further results at the workshop and in subsequent submissions. The analytical theorems (Section 1.2) and FPGA synthesis result (Section 1.3) do not depend on these measurements. The 98.4% MNIST accuracy (Table 2) is the recorded outcome of the reported run, but full reproducibility requires the pending configuration details and multi-seed evaluation.

These results demonstrate that a Cortex built entirely from Estimator primitives, receiving only short bit-pattern messages through a narrow channel, can learn to classify binarized MNIST at accuracy competitive with shallow neural baselines on the same task, while using only bit-comparison updates, integer-comparison splits, and integer argmax for prediction—no multiplications, no logarithms, no floating-point arithmetic.

4.10 Key Properties

- **Zero-knowledge startup:** every Estimator initializes at $s = 1$. Knowledge is built from nothing—no random weights.
- **Local learning:** every update is local to one node of one tree. No global loss function, no backward gradient propagation.

- **Stochastic exploration:** the Estimator’s inherent randomness provides natural exploration of the tree-structure space without an explicit exploration parameter.
- **Designed for incremental improvement without global reset:** REPROGRAM prunes only the directly-affected subtrees of the Cortex, leaving the remaining learned structure intact. Quantifying catastrophic-forgetting avoidance against a no-REPROGRAM baseline requires the ablation flagged in Section 4.9’s measurement gaps.
- **Cortex-side hardware simplicity:** the Cortex inference and update path requires only three operations—Estimator update (compare + conditional ± 1), subtraction (split decision), and comparison (argmax prediction). No logarithms, no exponentials, no matrix multiplication. The Sensor side performs additional bookkeeping (test scoring, retirement, reassignment), but its inner-loop per-sample test evaluation reduces to the same boolean primitives (single-bit probes, XORs, region thresholds) that the test library is built from—no arithmetic deeper than the boolean primitives themselves.
- **Communication efficiency:** each cross-die exchange is $O(|q| + r)$ bits regardless of model complexity, in contrast to $O(p)$ parameter-synchronization traffic for p -parameter neural models. Whether this is an absolute advantage depends on p , the training batch size, and the per-sample exchange count (Section 4.7).

References

- [1] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, pp. 379–423, 623–656, 1948.
- [2] J. Felix, “A hardware-native bit-serial learner with exact statistical structure,” in *Proc. MLArch-Sys Workshop, ISCA 2026*, 2026.
- [3] P. Flajolet and A. Odlyzko, “Singularity analysis of generating functions,” *SIAM J. Discrete Math.*, vol. 3, no. 2, pp. 216–240, 1990.
- [4] P. Flajolet and R. Sedgewick, *Analytic Combinatorics*. Cambridge University Press, 2009.